Automatic extraction of semantic AOIs from heterogeneous SVG visualizations

Documentation

Charbel ABOU KHEIR, Jean Paul AL AM, Arsène MALLET, Marie-José TANNOURY, Alain TRAD.

Under the supervision of Anne-Flore Cabouat.

Contents

Data

Implementation Prototype

Comment (by Arsène):
Below I described how I started prototyping the pipeline. To be honest I haven't thought about it much, everything was quite naive and to get a first architecture of the pipeline that we can build on and improve upon. The point was also for me to get familiar with some of the tools we'll probably use. But of course, since we haven't even discussed anything concrete yet, do not consider that anything below is definitive (as suggested by the term *prototype* 😄).

Parsing an SVG stimuli

The first approach considered is to build a simplified version of the tree obtained with Python's built-in XML API. Still with the idea of simplifying things at first, we remove from this tree the non-visual or unsupported (*yet*) elements (see below the *current* cleaned elements)

List of the cleaned elements:

- `clipPath` • `defs` • `desc` • `filter` • `linearGradient` • `marker` • `mask` • `metadata`
- `pattern` • `radialGradient` • `script` • `style` • `title`

AOI Extraction

After building the SVG tree, the next step to extracting the AOIs is to be able to know where they appear on the screen. To do that, a first approach is to compute a **bounding** for every element in the tree. This can be done *quite* easily using the tree structure itself. We first implement methods that compute the bounding for every SVG visual element (this is done easily for simple geometrical shapes such as `circle`, `rectangle` or `polygon`, and for the trickier shapes such as `path`, we'll discuss method later) and after that we go from the leaf of the tree — which are these visual elements — to the root, uniting the bounding of all the children node at every step.

This approach brings two questions:
- *What bounding method can we use ?*
- *Given this bounding method, how to compute a meaningful and efficient union ?*

For the bounding, an appropriate method seems to be using polygons. From that structure, we'll then be able to refine or roughen the level of precision of our extraction depending on our needs (polygon simplification algorithm: Ramer-Douglas-Peucker algorithm).

TODO: find an appropriate way to unite two boundings (especially when they're disjoints).

AOI Identification

TODO.

Miscellaneous

> **Comment (by Arsène):**
> This section is dedicated to add or specify some information that need to be known by the other, without really knowing where (or even if) they belong yet.

Data source to visualization type correspondance:

- `A-a-[1|2].svg` → map
- `A-b-[1|2].svg` → treemap
- `B-a-[1|2].svg` → scatterplot (*as a weird histogram*)
- `B-b-[1|2].svg` → scatterplot
- `C-a-[1|2].svg` → map
- `C-b-[1|2].svg` → calendar heatmap
- `D-a-[1|2].svg` → topological map
- `D-b-[1|2].svg` → graph